Three rules and one numerical constructor for iPRA

Uniform recursion, witness discharge, and a smaller strict checker

Research prepared for Vladimir Reshetnikov

11 November 2026

Source snapshot: `11814771b7490dfbea0ed5fcf0f5b23586334764`

Abstract

We give a smaller complete presentation of full first-order intuitionistic primitive recursive arithmetic. Numerical syntax has variables, zero, and one ternary recursor; successor is a special instance. The recursor and the logical binder each bind two numerical variables in their final child. There is one fixed reflexivity clause and three inference families: clause use, introduction, and substitution at witness discharge. Computation is part of canonical syntax and is included in the checker. We prove the recovery of equality, full intuitionistic logic, literal-equation and quantifier-free induction, and both interpretation round trips with iPRA. A direct compiler for primitive recursion avoids pairing.

The construction combines existing ideas in the inspected repository; its contribution is the resulting smaller presentation and its checked implementation, not priority for its individual ingredients. A separate refinement of the existing native checker removes a duplicate grammar traversal. All size comparisons include complete standalone programs. No absolute minimality theorem or new Lean/Rocq verification is claimed.

Contents

1 Result, comparison, and precise scope **2**

2 The entire primitive presentation **3**

3 Canonical syntax and implementation invariants **4**

4 Arithmetic definitions without a separate successor **6**

5 Recovering equality and induction **7**

6 Full intuitionistic logic is present **8**

7 From equation induction to iPRA **10**

8 Both interpretations and their composites **11**

9 The standalone certificate checker **12**

10 A separate smaller strict native checker **13**

11 Executed evidence, reproduction, and limits **14**

A Source map and attribution **15**

1 Result, comparison, and precise scope

The reference is `VladimirReshetnikov/ProveIt`, directory `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`, at the commit printed on the title page. We inspected its current overview, the relevant filed checkers and proofs, and all six incoming archives in that snapshot. In particular, the comparison includes `inferred_witness`, `deterministic_opening`, `clause_discharge`, `first_instance`, `prefix_clauses`, and `substitutive_discharge`. These are part of the baseline, even where the overview has not yet incorporated their results.

The mathematical simplification. The existing three-family scalar discharge presentation has distinct successor and iterator constructors. The presentation $\mathcal{R}_3$ uses a counter-aware recursor R whose zero case returns the successor of its initializer. It removes the separate successor constructor, and makes the binding convention of numerical recursion identical to that of the logical paired binder. The rule count stays at three. Full normalization is an explicit part of the specification and of the measured code.

The separate implementation simplification. The current native public-language checker performs one traversal to check grammar and another to check scope. Its existing depth counter already records the syntactic sort by parity. Fusing the checks removes the independent validator while preserving the entire certificate acceptance relation. This refinement implements the existing native calculus; it is not an alternative count for the new $\mathcal{R}_3$ checker.

Complete standalone module	Bytes	Tokens	AST	Gzip-9
Scalar Name control	1,983	663	751	812
Uniform recursion, new	1,950	662	741	806
Scalar Name, with fuel	2,169	711	810	880
Uniform recursion, with fuel	2,136	710	800	873
Current strict native control	780	360	409	458
Strict native, explicit fusion	685	340	385	409
Strict native, integrated cases	661	336	381	390
Compact control, different contract	508	267	302	330

Table 1: Matched complete-checker measurements. The compact row has a different public-validation contract.

All primary measurements use significant Python tokens, parsed AST nodes, and UTF-8 source bytes after removing actual docstrings and applying `ast.unparse`, followed by one LF. The complete module, including JSON parsing, validation, substitution, normalization, and the success interface, is counted. The common Python interpreter and standard library are not. Deterministic gzip sizes are informational. The scalar fuel control is a constructed matched control using the same budget instrumentation; only the unbounded scalar control is an unchanged upstream module.

The much shorter compact codec has a different public contract and no separate strict public-goal grammar validator. Its size is reported to prevent a false global-minimum claim. A small saving in the main matched comparison is evidence for this particular implementation, not a language-independent measure of mathematical simplicity.

Theorem 1.1 (Main result)**.** *The calculus $\mathcal{R}_3$ specified below is a faithful proof presentation of an ordinary intuitionistic theory bi-interpretable with first-order iPRA. Both interpretation domains are the whole numerical domain; equality is actual equality. In each direction the numerical composite is*

internally isomorphic to the identity by the graph $x = y$, and every formula is provably equivalent to its double translation. The systems are equiconsistent.

Assurance. The new metatheorems have written proofs in this paper and an independent mathematical review in the archive. Executable checks test the concrete implementations. They are not a proof-assistant verification. The repository's Lean results establish the inherited native/scalar bridge; we do not represent those results as a formalization of the new recursor normalizer or of this Python code.

2 The entire primitive presentation

2.1 Numerical syntax and canonical computation

There is one numerical sort. Raw terms are

$$t ::= x \mid 0 \mid R(n, a; i, u.h).$$

Only i, u are bound in h; they are not bound in n or a. All parameters of h other than i, u remain fixed during recursion. Define the notation

$$S(a) := R(0, a; i, u.0), \qquad \mathbf{1} := S0.$$

Thus S is not an additional term constructor. The computation equations are

$$R(0, a; i, u.h) = S(a), \tag{1}$$

$$R(Sn, a; i, u.h) = h[n/i, R(n, a; i, u.h)/u]. \tag{2}$$

Substitution on the right is simultaneous and capture avoiding.

Public terms are the full computation normal forms for these equations, written $\mathsf{N}(t)$. At a zero counter the body is represented canonically by 0, so $R(0, a; i, u.0)$ is the canonical successor node. A canonical nonzero recursion node has a counter that is neither zero nor successor headed. All its children, including its body, are canonical. All rule substitutions below include normalization. No formula binder is erased by normalization.

More explicitly, normalize the children n, a, h. If $n = 0$, return $R(0, a; i, u.0)$. If $n = R(0, m; i, u.0)$, normalize the predecessor recursion and then the simultaneous substitution in (2). Otherwise retain the recursion node. The zero-body erasure is performed after the supplied children have been checked; it is not a license to hide malformed syntax in an erased child.

2.2 Formulas, clauses, and falsity

Native formulas are

$$A ::= s = t \mid \mathsf{D}_{x,y}B(x, y).$$

The paired binder binds x, y throughout B. Its ordinary reading is

$$\mathsf{D}_{x,y}B(x, y) \;:=\; \exists x(B(x, 0) \to \forall y\, B(x, y)). \tag{3}$$

The designated contradiction is $\bot_* := (0 = \mathbf{1})$.

A context is a finite reusable list of clauses $[A; y.B(y)]$, read as $A \to \forall y B(y)$, where y is absent from A. An empty antecedent is allowed and is written $[\,; y.B(y)]$, meaning $\forall y B(y)$. An ordinary assumption F is the constant clause $[\,; v.F]$ for unused v. Every context contains the single fixed axiom clause

$$[\,; y.y = y]. \tag{4}$$

The initial closed-proof context consists of this clause alone.

2.3 Exactly three inference families

Use. For a retained clause $[A; y.B(y)]$ and any well-scoped term t,

$$\frac{\Gamma \vdash A}{\Gamma \vdash \mathsf{N}(B(t))}.$$

If the antecedent is empty, the rule has no premise. In particular, (4) derives every $t = t$.

Close. Let t be a term in the outer context and z fresh for that context, the conclusion, and t. Set

$$\eta = \begin{cases} z, & \text{if } B \text{ is a literal equation and } t \text{ is literally } 0, \\ 0, & \text{otherwise.} \end{cases}$$

Then

$$\frac{\Gamma, \mathsf{N}(B(t,\eta)) \vdash \mathsf{N}(B(t, Sz))}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{5}$$

The added assumption is an unconditional constant clause and is discharged. This is a single syntactically determined rule. It does not allow induction on arbitrary quantified formulas.

Name. From a proof of $\mathsf{D}_{x,y}B$, introduce a fresh numerical parameter b and the clause

$$\mathcal{B}_b = [\mathsf{N}(B(b,0)); y.\mathsf{N}(B(b,y))].$$

The rule is

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B \qquad \Gamma, \mathcal{B}_b \vdash b = t \qquad \Gamma, \mathcal{B}_b \vdash F}{\Gamma \vdash \mathsf{N}(F[t/b])}, \qquad b \notin \mathrm{FV}(\Gamma, \mathsf{N}(F[t/b])). \tag{6}$$

The replacement term t and intermediate formula F may mention b. Substitution replaces its free occurrences once and does not revisit an inserted copy of t. The exported normalized formula must be in the outer scope. The special case $t = b$ is permitted.

No other primitives. The syntax conventions, fixed clause, and these three families are the whole calculus. There is no primitive equality-transport rule, hypothesis rule separate from Use, computation inference, ordinary logical axiom, successor axiom, or separate induction rule. Computation and the normalization algorithm nevertheless count as trusted specification.

3 Canonical syntax and implementation invariants

3.1 Why normalization is legitimate

Lemma 3.1 (Normalization and substitution)**.** *Every well-scoped raw numerical term has a unique computation normal form. Normalization preserves scope and is compatible with renaming and capture-avoiding substitution:*

$$\mathsf{N}(\mathsf{N}(t)[\mathsf{N}(\sigma)]) = \mathsf{N}(t[\sigma]).$$

The same statements hold componentwise for formulas and clauses.

Proof. For the termination argument, temporarily use an auxiliary language with an independent successor and the ordinary base-type primitive recursor. Read $R(n, a; i, u.h)$ as recursion with initial value $S(a)$ and update $h(i, u)$. Its zero and successor reductions are the base-type fragment of the reductions of Gödel's system T, whose strong normalization is established by the usual reducibility

argument; a primary account of the surrounding typed normalization framework is cited in [7]. The ordinary recursive syntax here has only numerical variables and numerical results: higher-type recursion is not being added to the object theory.

One can see the relevant fragment of the argument directly. Declare a numerical expression computable when it is strongly normalizing; declare a fixed update computable when substituting computable numerical arguments gives a computable expression. Induction on term formation gives closure under the zero and successor constructors. For a recursor, use the reduction heights of its arguments and a subsidiary induction on the successor spine of a normal counter. At a neutral counter there is no head reduction. At a successor counter the recursive call has the shorter spine and the computable update preserves computability. The same argument under computable substitutions treats open bodies and their reductions. This is the standard base-type recursor case of the reducibility proof.

In the auxiliary translation, treat the designated canonical node $R(0, a; i, u.0)$ directly as $S(a)$; translate every other R node as the ordinary successor-seeded recursor. With this convention an erasure of another zero-body node is a zero reduction in the auxiliary syntax. If a reduction inside a body creates the canonical successor shape, follow its translated reduction by the corresponding zero reduction. Every genuine successor contraction is an ordinary recursor reduction. Changes inside a body later erased are already covered by strong normalization of that body. Thus the canonical notation does not introduce an infinite reduction.

The ordinary zero and successor rules have disjoint head patterns; reductions inside their arguments commute with the head reduction, using substitution in the successor case. Erasure of an unused body also commutes with reductions inside that body. These observations give local confluence. Termination and local confluence give unique normal forms. Reductions are stable under capture-avoiding substitution, so the two displayed terms are convertible and have the same unique normal form. Binder structure at formula nodes is fixed, so the formula and clause statements follow. □

No universal primitive-recursive normalizer is assumed. Each fixed term denotes a primitive recursive function. It does not follow that evaluation over arbitrary term descriptions is primitive recursive uniformly. Such a universal evaluator would yield the usual diagonal contradiction. The unbounded normalizer is an external total recursive procedure. The archive also supplies an explicitly fuel-bounded checker. Bounded simulation gives a primitive-recursive certificate predicate with the same set of provable formulas, since every finite successful run has some sufficient budget. No internal assertion that all description codes normalize is an axiom of $\mathcal{R}_3$ or iPRA.

3.2 A binding detail that matters

The concrete representation uses 0 for zero, negative integers for allocated free parameters, and positive one-based de Bruijn indices for bound variables. Terms are ternary arrays $[n, a, h]$, equations are binary arrays $[s, t]$, and D formulas are unary arrays $[B]$. For both the recursor body and the paired formula body, index 1 is the inner variable (u or y), and index 2 is the outer variable (i or x).

In a successor contraction, remove the counter binder first, under the retained state binder, and remove the state binder second:

$$\mathrm{sub}\big(\mathrm{sub}(h, n, \mathit{depth} = 1), \mathsf{N}(R(n, a; i, u.h))\big).$$

The depth-one insertion lifts the counter term correctly. Substituting the previous state first without lifting it would capture an ambient positive index under the still-present counter binder. Nested-binder regressions in the executable suite specifically check this case.

The checker synthesizes a formula at each proof node, then validates its canonical form and numerical scope. Name obtains its matrix from the major proof and its target from the equality proof; neither is

trusted merely because it appears in the certificate. An exported formula is checked in the outer scope, so witness escape is rejected. The public result is compared with the fully validated claimed goal.

4 Arithmetic definitions without a separate successor

All displayed definitions denote fixed finite terms; there is no quantification over function codes or an internal universal evaluator. Write equality of canonical syntax as $\equiv$.

Definition 4.1 (Predecessor on successors and selection)**.** Define

$$W(n) := \mathsf{N}(R(n, 0; i, u.i)), \tag{7}$$
$$C(n, a, b) := \mathsf{N}(W(R(n, a; i, u.Sb))), \tag{8}$$
$$P(n) := C(n, 0, W(n)), \tag{9}$$
$$K(y) := C(W(y), \mathbf{1}, 0). \tag{10}$$

The bound variables i, u in the selector are unused in its update.

Lemma 4.2 (Endpoint computations)**.** *The following are literal normal-form identities:*

$$W(0) \equiv \mathbf{1}, \qquad W(Sn) \equiv n,$$
$$C(0, a, b) \equiv a, \qquad C(Sn, a, b) \equiv b,$$
$$P(0) \equiv 0, \qquad P(Sn) \equiv n,$$
$$K(\mathbf{1}) \equiv \mathbf{1}, \qquad K(S\mathbf{1}) \equiv 0.$$

Proof. The successor equation for W returns its counter argument and ignores its state. The zero equation returns $\mathbf{1}$. In $C(0, a, b)$ the inner recursor returns Sa, to which W applies as a left inverse. In $C(Sn, a, b)$ the inner recursor returns Sb. The statements about P and K are then immediate substitutions. No induction, equality transport, or range theorem for K is used. □

Definition 4.3 (Ordinary initialized recursion)**.** For a fixed update $h(i, u)$ put

$$J(n, a; i, u.h) := \mathsf{N}\left(W\left(R(n, a; i, v.S(h(i, W(v))))\right)\right). \tag{11}$$

Lemma 4.4 (A direct primitive-recursion interface)**.** *For arbitrary terms and fixed passive parameters,*

$$J(0, a; i, u.h) \equiv a, \qquad J(Sn, a; i, u.h) \equiv \mathsf{N}(h(n, J(n, a; i, u.h))).$$

Proof. At zero the outer W cancels the successor of the initializer. At a successor the update returns the successor of $h(n, W(R(n, a; \ldots)))$; the outer W cancels that successor. Lemma 3.1 identifies the nested normalizations. □

Initial functions and composition are now immediate, and a primitive recursion $f(0, \vec{p}) = g(\vec{p})$, $f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$ compiles as

$$f^*(n, \vec{p}) = J(n, g^*(\vec{p}); i, u.h^*(i, u, \vec{p})).$$

This is external recursion on a finite PR description. Unlike the scalar iterator compiler, it requires no pairing, projection construction, or encoded clock. Each defining equation follows by literal normalization and reflexivity. The scalar iterator remains available as $I(n, a; u.h) := J(n, a; i, u.h)$ with i unused.

5 Recovering equality and induction

We give the proof because simply moving equality into witness discharge would otherwise leave an apparent circularity. The construction adapts the two-row packet of the existing `witness_discharge` proof [3]; its only numerical requirements are the endpoint identities of Lemma 4.2.

5.1 Structural rules, ordinary elimination, and cases

Weakening, exchange, contraction, injective renaming of free parameters, and cut of an ordinary assumption are admissible. Prove these by induction on finite derivations, freshening local parameters when needed. An ordinary assumption is used through its constant unconditional clause; replace each such use by the supplied derivation. No arbitrary numerical substitution theorem is needed for these structural facts.

Lemma 5.1 (Ordinary paired elimination)**.** *From* $\Gamma \vdash \mathsf{D}_{x,y}B$ *and* $\Gamma, \mathcal{B}_b \vdash A$*, with* $b \notin \mathrm{FV}(\Gamma, A)$*, one may infer* $\Gamma \vdash A$*.*

Proof. In Name take $t = b$. Its equality premise is reflexivity $b = b$, and its formula premise is A. The export is literally A. □

For an unused witness coordinate abbreviate $\mathsf{Q}_y A(y) := \mathsf{D}_{x,y}A(y)$.

Lemma 5.2 (Closure and numerical cases)**.** *The following rules are derived:*

$$\frac{\Gamma, A(0) \vdash A(Sz)}{\Gamma \vdash \mathsf{Q}_y A(y)}, \qquad \frac{\Gamma \vdash \mathsf{Q}_y A(y) \quad \Gamma \vdash A(0)}{\Gamma \vdash A(t)}.$$

Consequently $\Gamma \vdash A(0)$ *and* $\Gamma \vdash A(Sz)$*, with* z *fresh, imply* $\Gamma \vdash A(t)$*.*

Proof. Introduce the unused witness at $\mathbf{1}$, which is not literal zero, so Close selects cases. For elimination, open the paired formula using Lemma 5.1; its clause is $[A(0); y.A(y)]$. Use the supplied base proof and target t. For numerical cases weaken the successor proof by the unused base assumption and compose these two steps. □

5.2 Transport from one equality-bearing discharge

Lemma 5.3 (Term transport)**.** *For every fixed term template* $h(w)$*,*

$$\Gamma \vdash s = t, \quad \Gamma \vdash \mathsf{N}(h(s)) = r \quad \Longrightarrow \quad \Gamma \vdash \mathsf{N}(h(t)) = r.$$

Proof. Choose x, y fresh and set

$$B(x, y) := \big(C(y, 0, C(K(y), x, \mathsf{N}(h(x)))) = C(y, 0, C(K(y), t, r))\big).$$

To introduce $\mathsf{D}_{x,y}B$ at witness s, its successor row is $C(K(Sz), s, \mathsf{N}(h(s))) = C(K(Sz), t, r)$. Apply numerical cases to $C(v, s, \mathsf{N}(h(s))) = C(v, t, r)$ at $v = K(Sz)$. The zero case is $s = t$, and every successor case is $\mathsf{N}(h(s)) = r$. This ignores the temporary assumption of Close and therefore works in either selected mode, including $s = 0$.

Open the package at fresh b. Its zero row is $0 = 0$. Use its clause at $\mathbf{1}$ and $S\mathbf{1}$ to obtain respectively $\mathsf{N}(h(b)) = r$ and $b = t$. In Name use these as the formula and equality premises. The export is $\mathsf{N}(h(t)) = r$ by normalization– substitution coherence. None of the passive data contains b. □

Symmetry follows by taking $h(w) = w$ and $r = s$. Congruence follows using a reflexive minor premise; transitivity follows by joining at the root. Repeating transport gives replacement on both sides of an equation.

Lemma 5.4 (Formula transport)**.** *From $s = t$ and $F(s)$ one may derive $F(t)$ for every native formula template F.*

Proof. Induct externally on the number of paired binders. The atomic case is two applications of term transport. For $F(v) = \mathsf{D}_{x,y}B(v, x, y)$, eliminate $F(s)$ at a fresh c. Introduce $F(t)$ at witness c, which is not literal zero and so uses cases. Under $B(t, c, 0)$, the induction hypothesis and symmetry give $B(s, c, 0)$. Use the old clause at Sz, and transport its result to $B(t, c, Sz)$ by the induction hypothesis. Close and eliminate. Every recursive transport has a strictly smaller formula template. □

5.3 The missing zero-witness case is derived

Let $H = (0 = 0)$ and, for an equation $E = (s = t)$, put

$$G_v E := (C(v, 0, s) = C(v, 0, t)), \qquad L(E) := \mathsf{Q}_v G_v E.$$

The endpoints are $G_0 E \equiv H$ and $G_{Sw} E \equiv E$. Closure and Use show $E \dashv\vdash L(E)$ in every retained context.

Suppose $\Gamma, E(0, 0) \vdash E(0, Sz)$ is a case premise for an atomic matrix at witness zero. Apply the same premise, through the uniform equivalence just proved, to the compound matrix $A(x, y) = L(E(x, y))$. Close at witness zero now selects cases because A is compound. Open $\mathsf{D}A$ at fresh c; introduce $\mathsf{D}E$ at witness c, wrapping its base, using the opened clause, and unwrapping its successor row. This recovers the omitted atomic case introduction. It does not rely on a closed nonliteral term that normalizes to zero.

Corollary 5.5 (Unrestricted cases and equation induction)**.** *Case introduction is derived for every matrix and witness. In addition, for each literal equation $E(n)$ and arbitrary retained context,*

$$\Gamma \vdash E(0), \qquad \Gamma, E(z) \vdash E(Sz) \quad \Longrightarrow \quad \Gamma \vdash E(t).$$

Proof. For induction use Close in progress mode at witness zero with a matrix independent of its witness coordinate. This produces $\mathsf{Q}_y E(y)$, which Use specializes from the given base. □

The derived liberal calculus with ordinary elimination, equality transport, unrestricted case introduction, and equational progress is useful for translations: it avoids assuming that the literal-zero mode test is preserved by substitution. In particular, it handles translations whose normalized witness becomes zero.

6 Full intuitionistic logic is present

For completeness we specify the logical macros and their proof interfaces. These are abbreviations constructed from Eq and D; they are not additional primitive connectives or rules.

6.1 Gates, padding, and formula selection

First observe that paired binders preserve pointwise interderivability of their matrices in arbitrary retained contexts. To convert $\mathsf{D}B$ into $\mathsf{D}C$, open the former at a fresh b and introduce the latter at witness b, hence in case mode. Convert $C(b, 0)$ back to $B(b, 0)$, use the opened clause at Sz, and convert $B(b, Sz)$ forward to $C(b, Sz)$. The reverse direction is symmetric. This binder-congruence principle is a derived finite proof transformation.

Extend G_v recursively through paired binders, keeping v fresh for their bound variables. Then $G_0 A$ is derivable and $G_{Sw} A \dashv\vdash A$. At an equation these facts are endpoint computation and reflexivity. At

a binder, use unrestricted case introduction and ordinary elimination with the smaller-matrix facts. If the argument already depends on the control, read these statements pointwise: $G_0A(0)$ is derivable, and $G_{Sw}A(Sw) \dashv\vdash A(Sw)$. This convention is used in the conjunction and universal-quantifier macros below.

The padding operation $L(A) = \mathsf{Q}_v G_v A$ is equivalent to A and increases its number of D binders by one. Every native formula is a finite chain of paired binders ending in one equation. Pad two formulas to the same depth. Define $\mathrm{Sel}_v(A, B)$ by retaining their common binder prefix and using $C(v, -, -)$ on their two corresponding equation endpoints. Bound variables are renamed and shifted consistently before alignment. Computation and binder congruence give

$$\mathrm{Sel}_0(A, B) \dashv\vdash A, \qquad \mathrm{Sel}_{Sw}(A, B) \dashv\vdash B.$$

Numerical cases consequently give selection from two proofs and case elimination of a selected formula. This alignment construction also handles formulas of different initial depths; it does not assume that an unused constant paired binder preserves a formula.

6.2 The macros and all ordinary rule interfaces

Write $\exists^*_x A := \mathsf{D}_{x,y} G_y A(x)$. Then use the following abbreviations, with all displayed control variables fresh:

Ordinary operation	Native abbreviation
$\bot$	$0 = \mathbf{1}$
$A \to B$	$\mathsf{Q}_y \,\mathrm{Sel}_y(A, B)$
$A \wedge B$	$\mathsf{Q}_y G_y \,\mathrm{Sel}_{K(y)}(A, B)$
$A \vee B$	$\exists^*_v \,\mathrm{Sel}_v(A, B)$
$\exists x A(x)$	$\exists^*_x A(x)$
$\forall x A(x)$	$\mathsf{Q}_y G_y A(W(y))$

Lemma 6.1 (Natural deduction and formula meanings)**.** *These abbreviations admit every introduction and elimination rule of intuitionistic first-order natural deduction with equality, including the usual eigenvariable conditions and explosion. Under* (3)*, each abbreviation is equivalent to its ordinary logical meaning.*

Proof. For implication, closure introduction is implication introduction: the zero row is A, and the successor row is B. Use at $\mathbf{1}$ is modus ponens. For conjunction, the gate makes its zero row derivable. Its first two positive rows are B and A respectively because $K(\mathbf{1}) = \mathbf{1}$ and $K(S\mathbf{1}) = 0$. Conversely proofs of both conjuncts give the selector at every index by numerical cases, and then closure introduction gives the conjunction.

For $\exists^*$, a proof of $A(t)$ supplies every positive row of its gate, so case introduction gives the existential introduction rule. On elimination, the opened clause has a derivable zero row and yields $A(b)$ at $\mathbf{1}$; the fresh witness can then be discharged. Selection and this existential give both disjunction introductions and disjunction elimination.

For universal quantification the zero gate row is derivable and $W(Sz) = z$, so its positive row is $A(z)$. Generalization at a fresh z supplies all positive rows; specialization at St returns $A(t)$. The ordinary reading says exactly that every $A(t)$ holds, by these same endpoint identities and coverage.

From $0 = \mathbf{1}$, transport the control of $C(-, s, t)$ to obtain $s = t$ for arbitrary terms. Induction on formula depth with case introduction derives every native formula, giving explosion. Equality rules were derived in Section 5. All context-management steps use the structural admissibility proved there. Each displayed argument is intuitionistic and also proves the asserted equivalence under the ordinary reading. □

Numerical coverage. The derived numerical-case rule proves $t = 0 \vee \exists z\, t = Sz$: its zero case uses the left disjunct; its successor case uses the exhibited predecessor. Because $P(Sz) = z$, it also proves the stronger equation-based coverage

$$t = 0 \vee t = S(Pt). \tag{12}$$

This is a theorem on the whole numerical domain, not an appeal to the intended standard model.

7 From equation induction to iPRA

This section supplies the arithmetic step that cannot be replaced by claiming that primitive recursive functions have been represented. The reflection construction is adapted from the existing `equality_reflection.tex` proof [5]; its prerequisites are now available directly from R.

Work temporarily in the ordinary realization: intuitionistic first-order logic with actual equality, the R computation equations, coverage (12), separation $0 = \mathbf{1} \to \bot$, and contextual induction for literal equations. Successor is injective, since applying P to $Sx = Sy$ gives $x = y$. Put

$$\mathrm{sg}(x) = C(x, 0, \mathbf{1}), \qquad T(x) = C(x, \mathbf{1}, Px).$$

Coverage gives $\mathrm{sg}(x) = 0 \leftrightarrow x = 0$ and $\mathrm{sg}(x) = 0 \vee \mathrm{sg}(x) = \mathbf{1}$. It also gives $T(x) = 0 \leftrightarrow x = \mathbf{1}$. Applying sign to $Sx = 0$ yields $\mathbf{1} = 0$, so every successor is separated from zero. These arguments require no induction.

Using J, define addition and truncated subtraction by

$$x + 0 = x, \quad x + Sy = S(x + y), \qquad d(x, 0) = x, \quad d(x, Sy) = P(d(x, y)).$$

Literal-equation induction proves

$$0 + y = y, \qquad (Sx) + y = S(x + y),$$
$$d(Sx, Sy) = d(x, y), \qquad d(Sa, a) = \mathbf{1}.$$

For the cancellation identity induct on y with x fixed: its base is $P(Sx) = x$ and its step applies P to the previous identity. The diagonal identity then follows by induction on a.

Lemma 7.1 (Residual reflection)**.** *The ordinary realization proves*

$$d(Sa, b) = \mathbf{1} \;\leftrightarrow\; a = b.$$

Proof. First prove by equation induction on y, with x fixed,

$$C(\mathrm{sg}(d(x, y)), x, y + d(x, y)) = x. \tag{13}$$

At zero both selected branches are x. For the step put $r = d(x, y)$ and $r' = Pr = d(x, Sy)$. If $r' = 0$, the new selector immediately returns x. Otherwise coverage says r' is a successor. Coverage for r and $P0 = 0$ then imply $r = Sr'$; the induction equation gives $y + r = x$. The elementary addition identity $(Sy) + r' = y + Sr' = y + r$ gives the required selected branch. Every induction formula used here is a single equation; the case splits use only coverage.

Substitute $x = Sa$, $y = b$ in (13). If the residual is $\mathbf{1}$, then $b + \mathbf{1} = Sa$, hence $Sb = Sa$ and $b = a$ by predecessor. The converse follows from $d(Sa, a) = \mathbf{1}$ and equality replacement. □

Define

$$e(a,b) := \mathrm{sg}(T(d(Sa,b))).$$

The preceding results yield

$$e(a,b) = 0 \leftrightarrow a = b, \qquad e(a,b) = 0 \vee e(a,b) = \mathbf{1}. \tag{14}$$

These are internal statements about actual equality for all domain elements. In particular, equality is decidable; this has been derived without assuming excluded middle for arbitrary formulas.

For a fixed quantifier-free formula A, define its zero-true characteristic by external recursion:

$$\begin{aligned} b_{s=t} &= e(s,t), & b_{\bot} &= \mathbf{1}, \\ b_{A\wedge B} &= C(b_A, b_B, \mathbf{1}), & b_{A\vee B} &= C(b_A, 0, b_B), \\ b_{A\to B} &= C(b_A, b_B, 0). \end{aligned}$$

Using the two-valued ranges and a case split on b_A proves

$$b_A = 0 \leftrightarrow A, \qquad b_A = 0 \vee b_A = \mathbf{1}.$$

For implication, the zero branch of b_A gives A and hence reduces $A \to B$ to B; the one branch gives $\neg A$ and therefore the implication by explosion. This uses only the proved decidability of the finite characteristics.

Now translate the base and step of any quantifier-free induction instance to the equation $b_A(n) = 0$, apply equation induction, and translate back. The retained context may contain arbitrary quantified formulas. Thus the presentation recovers precisely the contextual quantifier-free induction of first-order iPRA.

8 Both interpretations and their composites

8.1 Ordinary realization and its exact boundary

Let T_R be ordinary intuitionistic first-order logic with actual equality and genuine falsity, the staged R descriptions and equations (1)–(2), coverage, separation $0 = \mathbf{1} \to \bot$, and contextual equation induction. Staging means that each recursion description has a fixed earlier-stage numerical update, with explicit finite parameters. There is no additional sort of functions.

The separation sentence must be included: the positive Eq/D formulas read in isolation admit singleton structures. The designated contradiction and full ordinary realization are part of the claim, so that this singleton issue is not silently discarded.

Lemma 8.1 (Native/ordinary adequacy)**.** *The native calculus $\mathcal{R}_3$ and T_R interpret one another faithfully under* (3) *and the logical macros, for arbitrary finite contexts. Their formula composites are interderivable.*

Proof. Read a native equation as equality and a native clause as its conditional universal meaning. Use is ordinary implication and universal elimination. In case-mode Close, retain its zero row and use coverage to obtain every row; then introduce the chosen witness. In progress-mode Close, use equation induction under the zero row. Name uses ordinary existential elimination and equality replacement in its branch; its exported result is fresh-witness free. Normalization replaces terms by provably equal terms: each finite reduction uses a computation equation; replacement inside a recursive body is justified by equation induction comparing the two recursive values.

Conversely, the logical rules are Lemma 6.1, coverage is (12), and equation induction is Corollary 5.5. Computation equations normalize to reflexive equations, and separation is the native explosion principle.

Each logical abbreviation has its claimed meaning. For a native paired binder, the expansion of $\exists x(B(x,0) \to \forall y B(x,y))$ implies $\mathsf{D}B$ by case introduction using the expanded witness and closure; the reverse implication opens $\mathsf{D}B$ and reconstructs that existential. Induction on the matrix gives the native formula round trip; induction on ordinary formula syntax gives the ordinary one. The same argument applies to clauses. □

8.2 The arithmetic interpretations

Interpret $R(n, a; i, u.h)$ in iPRA by the ordinary primitive recursion whose base is Sa and whose step is $h(i,u)$. The domain formula is $x = x$ and equality is unchanged. Its computation equations, coverage, separation and equation induction are iPRA theorems.

In the reverse direction, translate each finite PR description using the direct compiler in Lemma 4.4. The PR defining equations hold by computation. Section 7 supplies the successor axioms and all quantifier-free induction instances; the logical part is unchanged in the ordinary realization.

Lemma 8.2 (Internal uniqueness)**.** *Two fixed recursions with provably equal bases and pointwise equal updates are internally equal at every counter.*

Proof. Use literal-equation induction on the equality of their values, with all passive parameters fixed. The base uses the given base equality. At a successor, the induction equality and congruence identify the state arguments, and the assumed update equality identifies the new values. The same proof applies to J, R, and ordinary primitive recursion. □

Proof of Theorem 1.1. The two translations just specified are interpretations. Induct externally on a fixed finite numerical description. Its double translation satisfies its original defining equations, using the external induction hypothesis for its earlier descriptions. Lemma 8.2 proves that it equals the original operation at every internal argument. This is not merely agreement on standard numerals.

Both domains are the whole numerical domain, so $x = y$ is an internally total bijection preserving each interpreted function and actual equality. Formula composites follow from numerical uniqueness, Lemma 8.1, and external induction on formula syntax. Ordinary falsity maps to $0 = \mathbf{1}$, which the ordinary theory proves equivalent to falsity by separation and explosion. Consequently a finite contradiction proof in either presentation translates into one in the other. This gives equiconsistency, with no assertion that either theory proves its own consistency. □

9 The standalone certificate checker

The public input is a JSON object with exactly the two keys `goal` and `proof`. The goal is a closed canonical Eq/D formula. Acceptance means exit status zero and exactly `ACCEPT` on standard output. Exceptions reject, and no trust decision uses Python `assert`.

At context length n, the available free parameters are $-1, \ldots, -(n-1)$ and the next fresh parameter is $-n$. The initial context length is one because of the reflexivity clause. A proof packet is one of:

- `['u',j,t,q]`: use clause j at t; q proves its antecedent if it has one. An unconditional clause has no premise, and its unused payload is conventionally `null`.
- `['i',B,t,r]`: introduce $\mathsf{D}B$ at witness t; r is the branch proof under the automatically selected assumption.
- `['e',q,e,r]`: q proves the paired major, e proves $b = t$ in its opened context, and r proves F there. The rule synthesizes the export $\mathsf{N}(F[t/b])$.

The sole unconditional axiom is encoded by the initial clause `([], [1,1])`. Since equations and recursion terms have different arities, one recursive walker can validate and rebuild both, adding two binder slots only in the final child of a unary formula node or a ternary numerical node. This shared binding rule is the specific syntax simplification.

Theorem 9.1 (Checker correspondence)**.** *For a closed canonical goal A, the standalone $\mathcal{R}_3$ checker accepts a finite certificate exactly when that certificate describes a derivation of A using the three specified families. Every finite $\mathcal{R}_3$ derivation has an accepted certificate under the usual unbounded finite-storage idealization.*

Proof. Induct on active proof packets. The synthesized formula of each recursive call has been checked for grammar, canonicality, and scope. In Use, its term is checked before instantiation and any antecedent proof must synthesize exactly the retained antecedent. In Close, the supplied matrix and its witness are checked, the generated local assumption is exactly the rule's selected row, and the branch result must equal the generated successor row. In Name the major must be unary, the equality must be binary with left endpoint exactly the freshly allocated parameter, and the final scope check excludes that parameter from the normalized export. Lemma 3.1 and the counter-first binding convention identify the computations with the stipulated substitutions.

Conversely encode a finite derivation recursively. Use its rule data and induction certificates for its premises. The canonical and freshness side conditions make every check pass. There is no trusted proof producer, theorem table, or omitted validator. The same construction with a budget exceeding the finite run cost gives an accepted fuel-bounded certificate. □

The counted source and a commented explanation are included in the archive. The minimal proof is

```
{"goal":[0,0],"proof":["u",0,0,null]}
```

which derives $0 = 0$ from the fixed clause.

10 A separate smaller strict native checker

This section concerns the current native scalar codec, not $\mathcal{R}_3$. Its baseline is the incoming `deterministic_opening` native public-language checker. The following argument proves an exact implementation refinement without changing any axiom, rule, numerical interpretation, or certificate.

The existing arity walker increases its depth by $5 - \mathrm{arity}(a)$ at every list node. If a formula position has p protected numerical slots, its depth is $2p$; a term position has depth $3 + 2p$. Thus even depth means formula and odd depth means term. Public list arities are exactly

$$1 \quad \text{or} \quad 2 + (d \bmod 2).$$

At a numerical leaf the canonical public integer is either zero or an odd integer at least three. The existing interval test checks its scope. The list cases respectively mean paired binder/equation or successor/iterator, with the codec's existing protected-slot conventions unchanged.

Enable this grammar predicate only for the initial no-image scope traversal, then disable it before reading any proof instruction. Structural induction on the public tree proves that the fused traversal succeeds exactly when the old grammar test and old scope traversal both succeed, and returns the same tree. Once the flag is off, the old walker and every subsequent kernel transition are unchanged. Therefore the entire certificate translation in both directions is the identity, including administrative payload aliases and unused prefixes. Resource-exhaustion thresholds and failure diagnostics are not equated.

The selected version states its added predicate explicitly. The smaller optional variant integrates it into the existing cases and uses immediate failure of the absent-image operation for malformed values.

Its rejection does not rely on exhausting the Python recursion limit. Both are supplied and audited; the clearer version is the recommended strict-native implementation.

Traversal fusion already occurs conceptually in earlier `scope_kernel`, `validated_core`, and related packages. The new claim is the specific measured refinement of the latest native checker, not priority for the idea or a new mathematical axiomatization.

11 Executed evidence, reproduction, and limits

The main implementation suite passed all of the following:

Check category	Passed
Accepted closed certificates	44
Separate-dispatch goal-directed replays	44
Translated existing zero-normal certificates	36
Malformed or unsound documents rejected	25
Malformed fuel annotations rejected	8
Exact-fuel and one-unit-insufficient checks	88
Ordinary/optimized Python CLI checks	60
Independent numerical comparisons	846
Generated normalization-idempotence checks	250
Selector and direct-recursion computations	60

The suite also detects the state-first capture mutant and accepts a sequent with two distinct free numerical parameters. Its goal-directed replay has separate proof-rule dispatch but shares the term normalizer; the numerical evaluator is independent of that normalizer.

One inherited fixture changes its witness to literal zero on normalization. The fixture translator deliberately declines that case. A separate direct proof of the same final formula is included. No universal executable proof compiler is claimed by this finite translation test.

The separate native refinement audit passed 143,580 generated syntax test positions, evaluated against three programs (430,740 program checks), with zero mismatches and zero recursion-limit rejection outcomes. Some random positions can repeat a value; these are not claimed to be that many distinct formulas. Its 18 pinned native proof fixtures passed their declared expected outcomes in all three programs under ordinary and optimized Python, for 108 CLI checks.

The main suite checks actual theorem certificates, malformed and unsound-input attacks, binding coherence, independent arithmetic semantics, and the bounded checker. The syntax audit for the separate native refinement uses an independently written decoder whose state is the number of protected slots, not the kernel's affine depth expression. Finite testing supports the implementations; the preceding proofs supply the universal claims.

Run `python3 reproduce.py` from the extracted directory to regenerate the validation and whole-program measurements. Build the paper with `python3 build_pdf.py`; a normal TeX Live installation is required only for the PDF. The acceptance modules use only Python's standard library and make no network calls.

What is new. The numerical grammar and the three-family discharge presentation are composed with one common two-variable binding convention, and the recursor-specific selector and direct PR compiler prove that the composition still has the required strength. The full normalizer, its capture-avoiding implementation, and the matched size comparison are part of that result. The separate native validator refinement establishes a larger source-size saving for an unchanged theory.

What is inherited. The paired-binder reading, witness-selected introduction, three-family Name interface, two-row transport construction, compound-envelope repair, logical gate constructions, and the equation-reflection argument come from the inspected research series. Counter-aware successor-seeded recursion and uniform binding also occur separately in that series. They are credited where used. This work claims a reduction by composition and implementation, not independent discovery of those ingredients.

What remains unproved. There is no global lower bound for a complete iPRA checker, no absolute simplest-axiomatization theorem, and no new Lean or Rocq verification of the delivered programs. The transparent arithmetic construction does not beat the compact codec in bytes. Normalization is fully counted, but concrete Python resource limits can still reject a valid very large proof. These limitations do not change the stated whole-domain bi-interpretation theorem.

A Source map and attribution

The exact repository snapshot is

`https://github.com/VladimirReshetnikov/ProveIt/tree/11814771b7490dfbea0ed5fcf0f5b23586334764/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`.

The archive's provenance file records the exact upstream paths, archive member names, and hashes. Frozen comparison modules and the inherited license are included. The unchanged scalar `output_control.py` from `ipra_inferred_witness` is the tightly matched control for the main checker. The strict native control comes from `ipra_deterministic_opening`. The latter archive's compact checker is reported separately.

For the inherited formal bridge, the relevant verified theorem statements are `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff` in `Lean/ConciseIPRA/NativeBiInterpretation.lean`. These statements are included as attributed references only; the upstream Lean build was not rerun for this package.

References

[1] V. Reshetnikov and the ProveIt research contributions, *Concise presentations of intuitionistic primitive recursive arithmetic*, repository snapshot identified above, 2026.

[2] ProveIt research packages, *Uniform binding*, *Counter evaluation*, and *Canonical recursors*, under `Research/` in [1].

[3] ProveIt research package, *Witness discharge*, `Research/witness_discharge/report.tex`, especially the ordinary-elimination, two-row transport, and compound-envelope arguments, in [1].

[4] ProveIt incoming packages, *iPRA with inferred witnesses*, *Deterministic witness opening for iPRA*, and the other four incoming archives enumerated in Section 1, `docs/incoming/`, in [1].

[5] ProveIt inherited arithmetic proof, *From literal-equation induction to full intuitionistic PRA*, `Research/witness_discharge/reference/equality_reflection.tex`, in [1].

[6] ProveIt, `Lean/ConciseIPRA/NativeBiInterpretation.lean`, `NativeLogic.lean`, and `NativeClausal.lean`, in [1].

[7] S. Urciuoli, *A Formal Proof of the Strong Normalization Theorem for System T in Agda*, 2023, arXiv:2303.13258. `https://arxiv.org/abs/2303.13258`. The standard typed-recursor normalization background is used only in the external normalization metatheory.